

HDOC – Day 4 (C Programming)

Experiment: Multiplication of Two Integers Without Using the Multiplication Operator (*)

Aim: To find the product of two integers without using the multiplication operator (*) by using repeated addition. The solution should use the minimum possible number of additions and should correctly handle positive numbers, negative numbers, and multiplication by zero.

Problem Statement

The given algorithm and flowchart are incorrect. We need to identify the mistakes, redesign the algorithm, draw a corrected flowchart, and verify the logic using the provided test cases.

Requirements

- Positive × Positive
- Positive × Negative
- Negative × Positive
- Negative × Negative
- Multiplication by Zero
- Minimum number of additions

Analysis of Given Algorithm

Mistakes Found:

1. The algorithm assumes that num2 is always positive.
2. It does not handle negative values correctly.
3. Multiplication by zero is not checked separately.
4. The sign of the final answer is never determined.
5. The algorithm may perform unnecessary additions when one number is much larger than the other.
6. Cases such as (-9×-4) and (6×-5) will not produce correct results.

Corrected Algorithm

1. Start.
2. Input two integers A and B.
3. If $A = 0$ or $B = 0$, print Product = 0 and stop.
4. Determine the sign of the result.
 - If one number is negative and the other is positive, sign = -1.
 - Otherwise sign = +1.
5. Convert both numbers into their absolute values.
6. To minimize additions, if $A < B$ then swap A and B.
7. Set Product = 0.
8. Repeat B times: Product = Product + A.
9. If sign = -1, Product = -Product.
10. Display Product.
11. Stop.

Corrected Flowchart (Text Form)

Start

↓

Input A, B

↓

Is $A = 0$ OR $B = 0$?

■ Yes

No ■

Print 0

Determine Sign

↓

↓

Stop

Convert to Absolute Values

↓

Swap if required (for minimum additions)

↓

Initialize Product = 0

↓

Repeat Product = Product + A, B times

↓

Apply Sign

↓

Print Product

↓

Stop

Explanation

The corrected approach uses repeated addition. To reduce the number of additions, the larger number is added repeatedly only as many times as the smaller number. The sign is applied at the end, which allows the loop to work only with positive values.

Verification Using Test Cases

$12 \times 3 = 36 \rightarrow 3$ additions

$3 \times 12 = 36 \rightarrow 3$ additions

$-6 \times 5 = -30 \rightarrow 5$ additions

$6 \times -5 = -30 \rightarrow 5$ additions

$-9 \times -4 = 36 \rightarrow 4$ additions

$-4 \times -9 = 36 \rightarrow 4$ additions

$0 \times 8 = 0 \rightarrow 0$ additions

$8 \times 0 = 0 \rightarrow 0$ additions

$0 \times -7 = 0 \rightarrow 0$ additions

$-7 \times 0 = 0 \rightarrow 0$ additions

$0 \times 0 = 0 \rightarrow 0$ additions

$1 \times -7 = -7 \rightarrow 1$ addition

$-1 \times -9 = 9 \rightarrow 1$ addition

Conclusion

The original algorithm was unable to handle negative numbers, zero values, and minimum-addition optimization. The corrected algorithm successfully computes the product of any two integers using repeated addition while performing the minimum possible number of additions. All the given test cases have been verified successfully.